ESCOGE TU PROPIA AVENTURA

Obligatorio de Programación 3 · ORT Uruguay

Documentación técnica y funcional de las aplicaciones de administración y lectura.

Integrante(s)	Número(s) de estudiante	Foto(s)
[COMPLETAR ANTES DE ENTREGAR]	[COMPLETAR ANTES DE ENTREGAR]	[INSERTAR FOTO(S)]

Versión: noviembre de 2026. **Nota:** este documento acompaña el código entregado. Los campos personales, capturas verificables, enlace de IA, archivo Astah, repositorio institucional y enlaces de SOMEE requieren completar la entrega.

Índice

Índice 2

1. Introducción y alcance 3

2. Diagrama de casos de uso 4

3. Narrativa: leer una historia (RF07) 4

4. Narrativa: historial de lecturas (RF08) 5

5. Diseño de casos de prueba: RF09 6

6. Diagrama UML: lógica de negocio 7

7. Diagrama UML: capas de lectura 10

8. Endpoints de la Web API y autorización 11

9. Precarga de datos con asistencia de IA 11

10. Investigación de estilos de lectura (RF12) 12

11. Preparación para la entrega y despliegue 12

1. Introducción y alcance

La plataforma permite a administradores gestionar usuarios, historias, capítulos y auditorías (RF01 a RF05). Los usuarios lectores pueden iniciar sesión con JWT, leer aventuras ramificadas, consultar su historial mensual y seleccionar estilo visual (RF06, RF07, RF08, RF12). Los administradores pueden consultar métricas de lecturas completadas (RF09 a RF11).

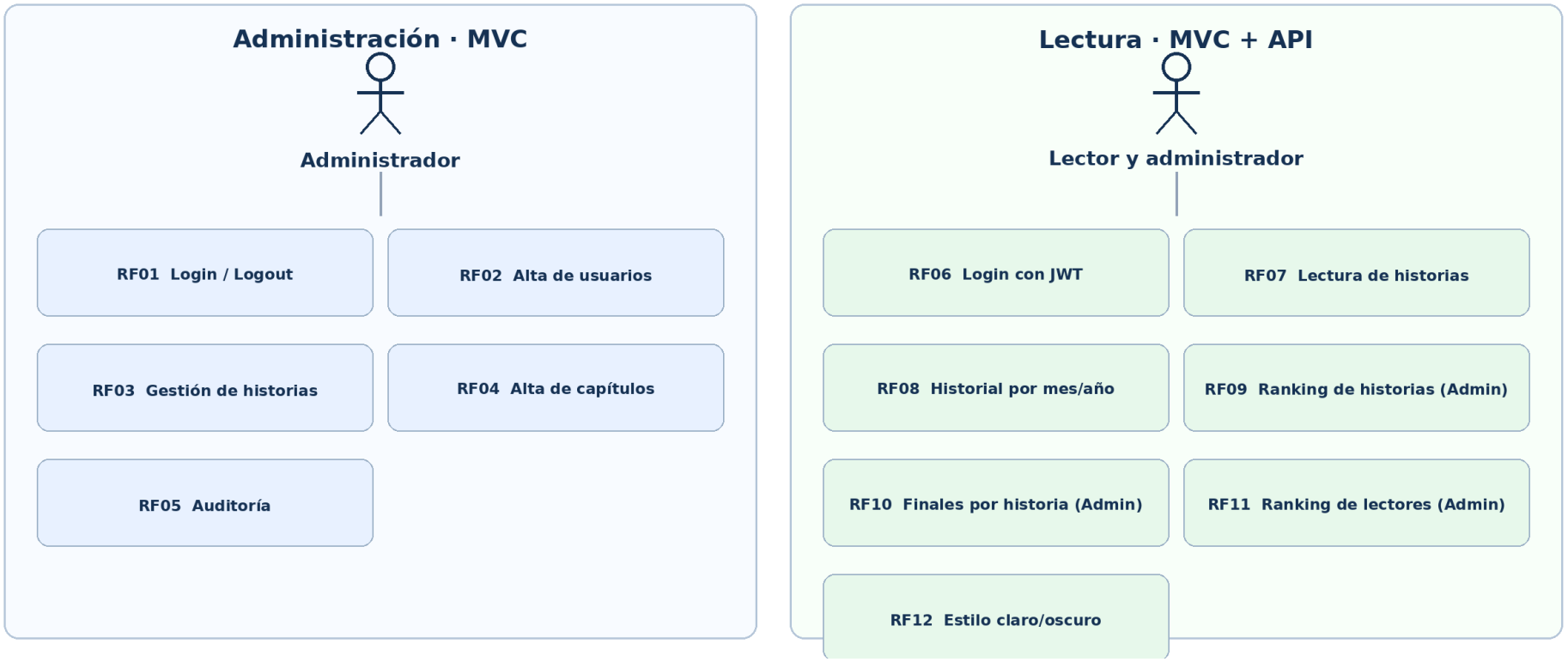
Cada historia tiene categorías, capítulos intermedios con opciones y capítulos finales. Una lectura registra al usuario, historia, comienzo, capítulo actual, final alcanzado y fecha de finalización cuando corresponde.

Soluciones y arquitectura

Solución	Proyectos	Responsabilidad
Obligatorio.Administracion.slnx	Obligatorio.Web + Lógica de Negocio + Acceso a Datos	MVC de administración, sesión local y ABM
Obligatorio.Lectura.slnx	Obligatorio.Api + Obligatorio.WebLectores + bibliotecas compartidas	Endpoints REST/JWT y cliente MVC con HttpClient

Ambas aplicaciones reutilizan las bibliotecas de negocio y datos, sin duplicarlas físicamente. Los Controllers de la API invocan repositorios directamente: es una arquitectura en capas funcional, pero todavía no aísla todos los casos de uso en una capa de aplicación independiente. Es una limitación respecto a una arquitectura Clean estricta.

2. Diagrama de casos de uso



Los actores Administrador y Lector utilizan RF06 en la aplicación de lectura. Los rankings RF09, RF10 y RF11 quedan restringidos al rol Administrador.

3. Narrativa: leer una historia (RF07)

Campo	Descripción
Actor principal	Lector autenticado
Precondiciones	JWT válido con rol Lector. La historia está publicada y tiene capítulo inicial intermedio.
Disparador	El lector selecciona Comenzar aventura.
Resultado	Se registra una lectura por recorrido. Al llegar al final se almacena fecha, final alcanzado y se actualiza el contador.

Flujo principal

- 1. El lector ingresa en la aplicación MVC y obtiene un JWT mediante POST /api/Auth/login.
- 2. MVC solicita GET /api/Historias y permite filtrar opcionalmente por categoría.
- 3. El lector selecciona una historia publicada.
- 4. MVC solicita POST /api/Lecturas/iniciar/{historiaId} con Authorization: Bearer.
- 5. La API registra Lectura con el lector del token, la historia, fecha y capítulo inicial.
- 6. MVC obtiene GET /api/Lecturas/{lecturaId}/actual y presenta texto y opciones.
- 7. El lector elige una opción. MVC envía POST /api/Lecturas/{lecturaId}/elegir/{opcionId}.
- 8. Se verifica propiedad de lectura, capítulo actual, opción válida y destino de la misma historia.
- 9. Si el destino es intermedio, se actualiza CapituloActualId y el proceso continúa.
- 10. Si el destino es final, se guarda FechaFinalizacion y FinalAlcanzado y se incrementa su contador.

Flujos alternativos y excepciones

- JWT ausente, inválido o vencido: HTTP 401; el MVC solicita nuevo login.
- Lector intenta abrir lectura de otra persona: HTTP 403.
- Historia o lectura inexistente: HTTP 404.
- Historia en BORRADOR u opción que no pertenece al capítulo actual: HTTP 400.
- Intento de continuar una lectura ya finalizada: HTTP 400. Para otro camino se inicia una nueva lectura.

4. Narrativa: historial de lecturas (RF08)

Campo	Descripción
Actor principal	Lector autenticado
Precondiciones	JWT con rol Lector, mes válido (1 a 12) y año (1900 a 2100).
Disparador	Abre Mis lecturas y elige mes/año.
Resultado	Solo se muestran lecturas del usuario autenticado que comenzaron en ese mes y año.

- 1. El MVC obtiene el token de la sesión y el mes/año seleccionados.
- 2. Envía GET /api/Lecturas/historial?mes=M&anio=A con el JWT.
- 3. La API identifica al usuario desde el token y consulta Lecturas mediante LINQ to Entities.
- 4. Devuelve la historia, inicio, finalización y final alcanzado cuando corresponde.
- 5. El MVC muestra lecturas finalizadas/en curso, con opción de continuar las que estén en curso.
- 6. Si no existen resultados, muestra “No hay lecturas registradas para el mes y año seleccionados”.

5. Diseño de casos de prueba: RF09

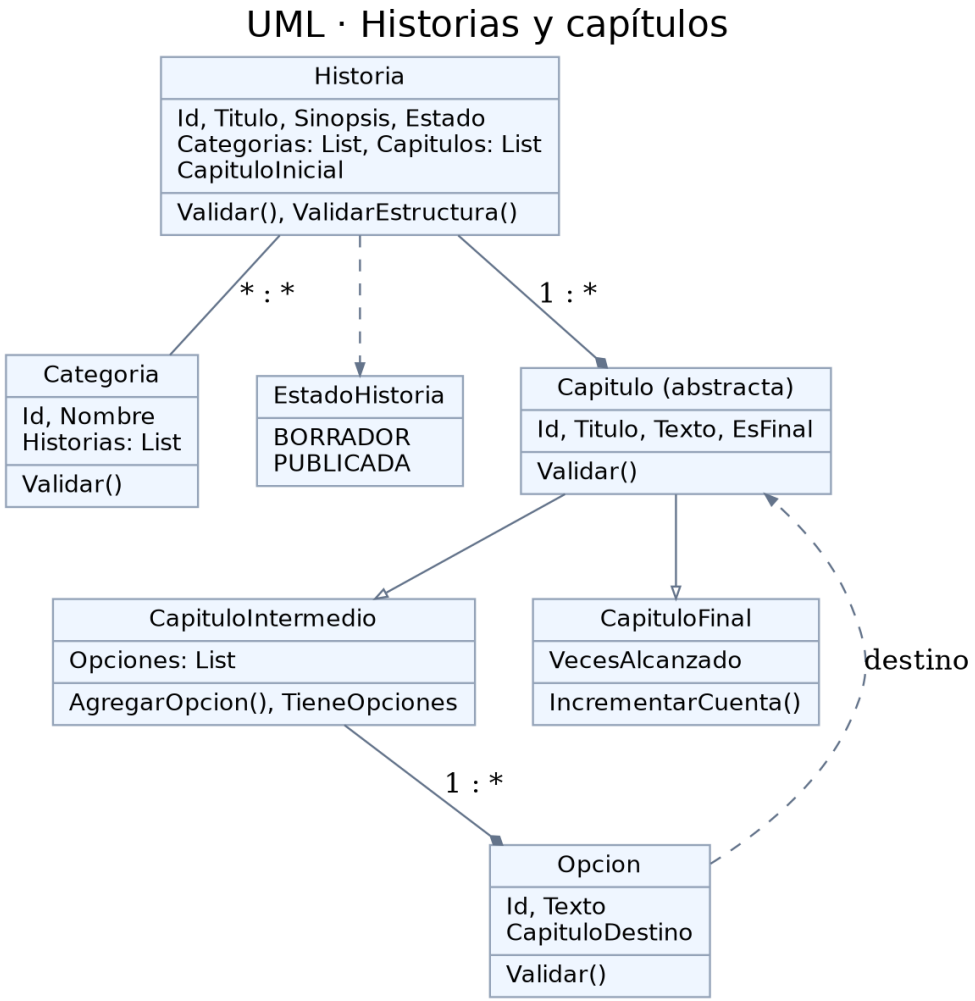
Ranking de historias más leídas: únicamente historias con al menos una lectura completada, ordenadas por cantidad de finalizaciones de mayor a menor. Las lecturas iniciadas sin finalización no cuentan.

Caso	Datos de entrada	Resultado esperado
CP01	Sin lecturas en la base	Ranking vacío; mensaje descriptivo.
CP02	Historia A con 3 lecturas iniciadas, 0 terminadas	A no aparece.
CP03	A: 4 terminadas; B: 2 terminadas	A antes que B.
CP04	A: 1 terminada + 5 en curso; B: 2 terminadas	B antes que A.
CP05	A y B: 3 terminadas cada una	Empatan en frecuencia y se aplica segundo orden por título.
CP06	Mismo usuario completa la misma historia 3 veces	Las 3 lecturas completas cuentan para RF09.
CP07	Lectura incompleta de A, luego completada	A pasa al ranking cuando se registra la finalización.
CP08	Usuario con rol Lector solicita /api/Estadisticas/historias	HTTP 403; solo Administrador accede.

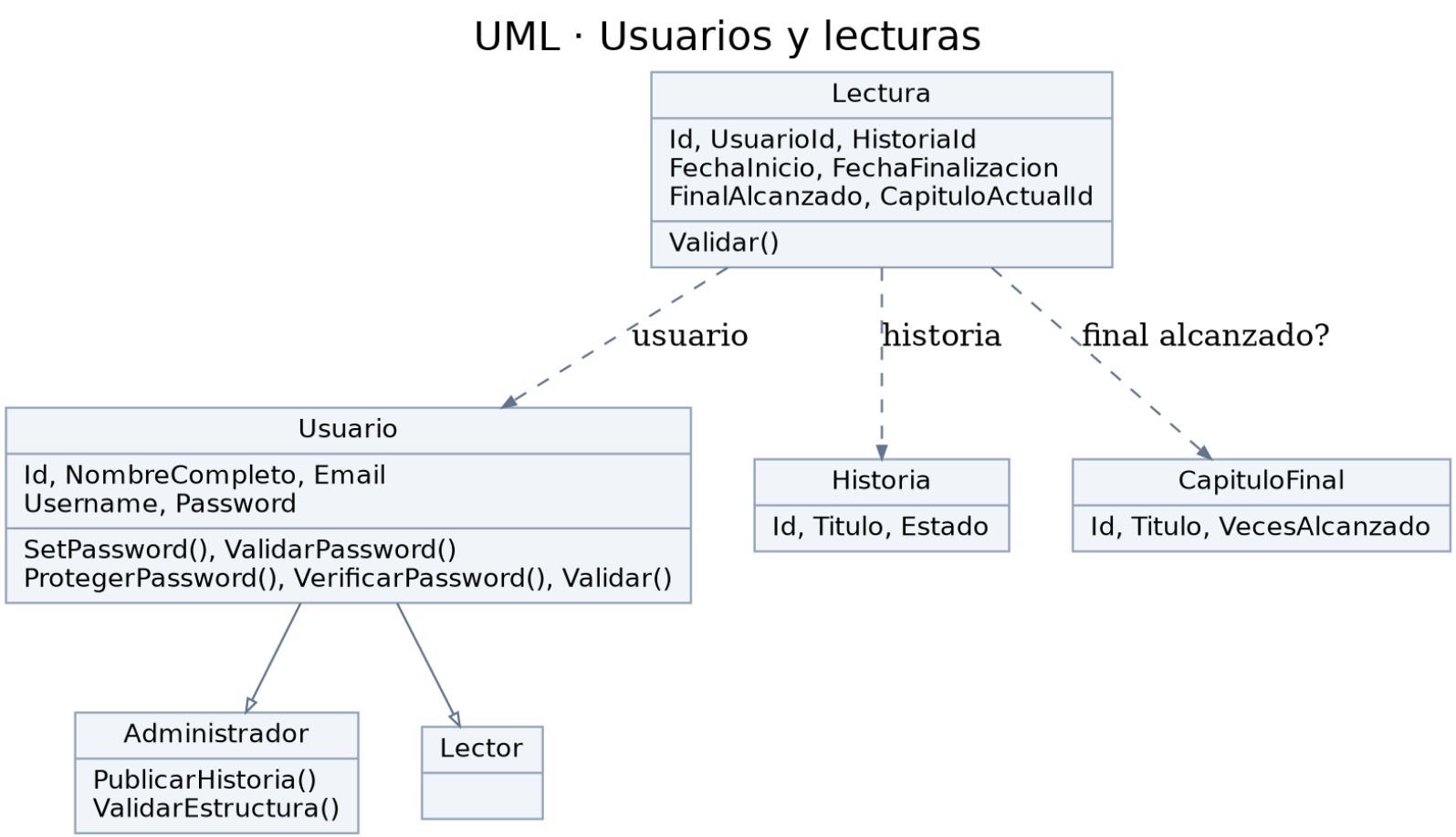
Datos de prueba: DatosIniciales.sql incluye frecuencias distintas para las historias P3 Demo. Las expectativas deben contrastarse con Swagger y consultas SQL ejecutadas en la instalación del equipo.

6. Diagrama UML: lógica de negocio

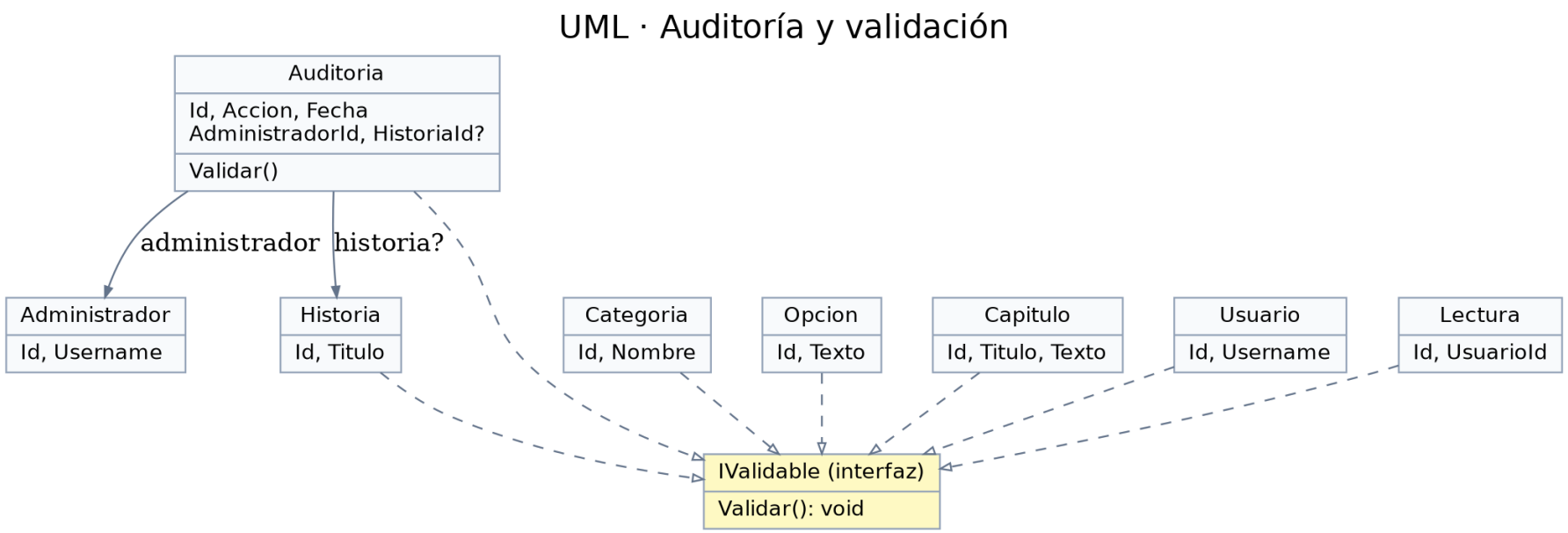
Se presenta el modelo completo en tres paneles legibles, con algunas clases repetidas como referencia de las relaciones entre paneles.



Panel 1. Categorías, historias, capítulos y opciones.

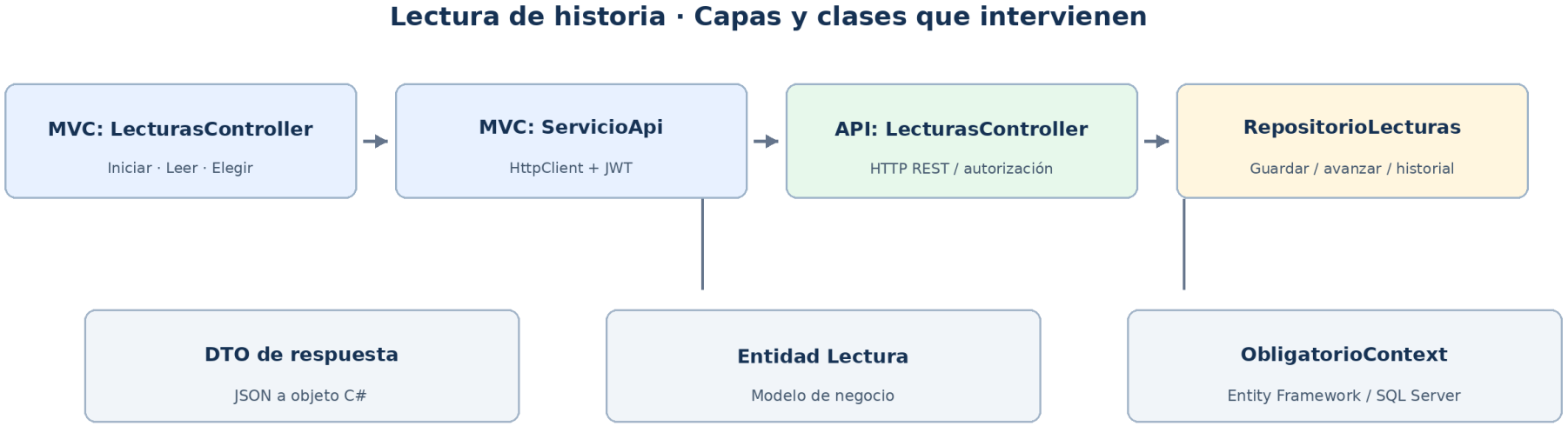


Panel 2. Usuarios, especializaciones y lecturas.



Panel 3. Auditoría, administrador e interfaz IValidable implementada por las entidades.

7. Diagrama UML: capas de lectura



Se incluyen únicamente los componentes de otras capas que participan en la funcionalidad de lectura. Las vistas .cshtml no figuran en este UML.

8. Endpoints de la Web API y autorización

Verbo	Ruta	Rol	Descripción
POST	/api/Auth/login	Administrador / Lector	Devuelve JWT y rol.
GET	/api/Historias?categoriaId=	Lector	Historias publicadas y filtro de categoría.
GET	/api/Historias/categorias	Lector	Catálogo de categorías.
POST	/api/Lecturas/iniciar/{historiaId}	Lector	Registra nueva lectura.
GET	/api/Lecturas/{lecturaId}/actual	Lector	Capítulo y opciones disponibles.
POST	/api/Lecturas/{lecturaId}/elegir/{opcionId}	Lector	Avanza y registra final.
GET	/api/Lecturas/historial?mes=&anio=	Lector	Lecturas del mes/año elegido.
GET	/api/Estadisticas/historias	Administrador	RF09.
GET	/api/Estadisticas/historias-disponibles	Administrador	Opciones del selector RF10.
GET	/api/Estadisticas/finales/{historiaId}	Administrador	RF10 y finales más frecuentes.
GET	/api/Estadisticas/lectores	Administrador	RF11.

Swagger está habilitado en modo Development (ruta /swagger). Los endpoints protegidos usan el encabezado HTTP Authorization: Bearer [JWT].

9. Precarga de datos con asistencia de IA

DatosIniciales.sql genera diez historias P3 Demo, cuarenta capítulos (dos intermedios y dos finales por historia), treinta opciones, diez categorías, diez o más usuarios, veinte registros de auditoría y lecturas con distintos resultados, incluidas algunas no finalizadas. La estructura permite probar búsquedas, métricas y selección de finales con frecuencias diferentes.

Secuencia de prompts documentada

• Prompt 1: “Generá historias de aventura con títulos y sinopsis distintos, aptas para diez categorías”.

• Prompt 2: “Para cada historia creá un capítulo inicial, otro intermedio y dos finales; conectalos mediante tres opciones para que todos los intermedios tengan salida”.

• Prompt 3: “Creá datos de prueba reproducibles para usuarios, lecturas completadas y en curso, con frecuencias diferentes para los rankings RF09-RF11”.

• Prompt 4: “Convertí los datos a SQL Server con Identity, transacciones, claves foráneas y sin IDs insertados manualmente”.

Captura y enlace verificable de la conversación de generación de datos: [COMPLETAR CON LA CAPTURA Y EL ENLACE REAL DE CHATGPT]. No se inventa un enlace ni se afirma que existe una captura adjunta.

10. Investigación de estilos de lectura (RF12)

Se investigó una implementación sin librerías adicionales mediante Bootstrap 5 y su atributo data-bs-theme. Bootstrap ya estaba incluido en el proyecto MVC y admite temas light/dark. Se eligió este enfoque porque reutiliza estilos de formularios, botones, tablas y tarjetas y reduce CSS duplicado.

Decisión implementada

• La vista Preferencias permite seleccionar claro u oscuro.

• El Controller valida que Estilo sea claro u oscuro y guarda el valor en una cookie con sufijo del UsuarioId (por usuario en el mismo navegador).

• Al ingresar, el MVC recupera la cookie asociada al lector y copia el estilo a su sesión.

• El layout aplica data-bs-theme="light" o data-bs-theme="dark" en la etiqueta html.

• Las cookies son HttpOnly, SameSite=Lax y Secure cuando la conexión es HTTPS.

Limitación asumida: la elección persiste en ese navegador, pero no se sincroniza automáticamente entre dispositivos. Esta decisión conserva el esquema actual de SQL Server sin agregar una tabla de preferencias no exigida explícitamente.

Registro de consulta a IA

Consulta trabajada en el proyecto: “Implementá dos estilos de lectura en MVC con Bootstrap sin añadir bibliotecas nuevas y conservando las preferencias por usuario en el navegador”. Falta incorporar a la entrega la captura real y el enlace compartido si la cátedra lo requiere.

11. Preparación para la entrega y despliegue

Abrir las soluciones Obligatorio.Administracion.slnx y Obligatorio.Lectura.slnx. Sobre SQL Server local: aplicar las migraciones de EF desde Obligatorio.Web y luego ejecutar DatosIniciales.sql. Como alternativa para una base nueva sin migraciones, utilizar EsquemaBD.sql seguido de DatosIniciales.sql. No mezclar los dos métodos en la misma base.

Para desplegar en SOMEE, cargar la cadena de conexión en la variable de entorno ConnectionStrings__Obligatorio y una clave secreta fuerte en Jwt__Key en el proceso de la API; publicar los sitios MVC de administración, Web API y MVC de lectores y configurar Api:BaseUrl en el cliente. No incluir claves de producción en archivos versionados.

Entregable solicitado	Estado / tarea final
Código + soluciones + API Swagger	Incluido, requiere compilación y ejecución en Visual Studio/.NET 10.
Esquema e inserts de prueba	EsquemaBD.sql + DatosIniciales.sql.
PDF con narrativa, casos, UML y prueba	Incluido, requiere portada personal y capturas reales.
Archivo Astah con diagramas	Pendiente: recrear/exportar los diagramas del PDF en Astah.
Repositorio GitHub institucional	Pendiente: agregar enlace y push desde la cuenta del grupo.
SOMEE + enlaces públicos	Pendiente: credenciales y publicación reales desde el entorno del grupo.

Enlaces de despliegue: Administración [COMPLETAR] · API [COMPLETAR] · Lectores [COMPLETAR] · GitHub [COMPLETAR]

Pruebas recomendadas antes de entregar: restaurar NuGet; compilar las dos soluciones; probar RF01-RF05 en la Web de administración; probar login JWT con lector y administrador; recorrer dos finales de una historia; consultar historial con y sin datos; comprobar que lector recibe 403 al solicitar métricas; confirmar los rankings con los datos SQL y el estilo claro/oscuro.